

隐马尔科夫模型

时间：2025年5月

TRANSWARP
星环科技

The background features a complex network diagram with numerous nodes (dots) and connecting lines (edges). The nodes are arranged in a way that suggests a spherical or globe-like structure, with lines crisscrossing between them. The overall color scheme is light gray and white, giving it a technical and modern feel.

目录

CONTENTS

- 1 贝叶斯网络
- 2 隐马尔可夫模型

1
chapter

贝叶斯网络

贝叶斯网络

- 把某个研究系统中涉及到的**随机变量**，根据是否条件独立绘制在一个有向图中， 就形成了贝叶斯网络。
- 贝叶斯网络(Bayesian Network)，又称**有向无环图模型**(directed acyclic graphical model, DAG)，是一种**概率图模型**，根据概率图的拓扑结构，考察一组随机变量 $\{X_1, X_2, \dots, X_n\}$ 及其N组条件概率分布 (Conditional Probabililty Distributions, CPD) 的性质

贝叶斯网络

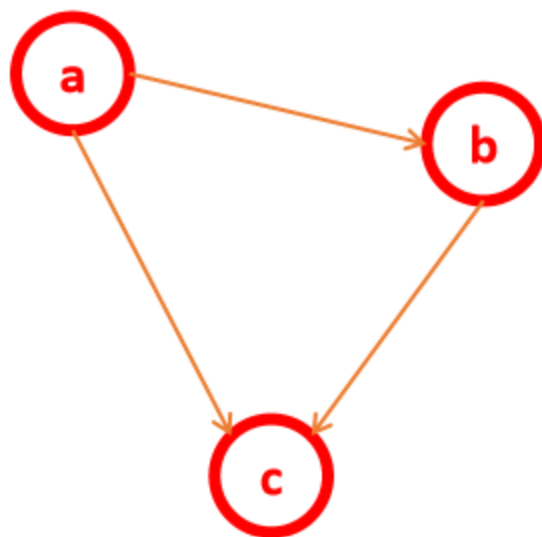
- 当多个特征属性之间存在着某种相关关系的时候，使用朴素贝叶斯算法就没法解决这类问题，那么贝叶斯网络就是解决这类应用场景的一个非常好的算法。
- 一般而言，贝叶斯网络的有向无环图中的节点表示随机变量，可以是可观察到的变量，或隐变量，未知参数等等。连接两个节点之间的箭头代表两个随机变量之间的因果关系(也就是这两个随机变量之间非条件独立)，如果两个节点间以一个单箭头连接在一起，表示其中一个节点是“因”，另外一个“果”，从而两节点之间就会产生一个条件概率值。
- 注意：每个节点在给定其直接前驱的时候，条件独立于其后继。

贝叶斯网络

- 贝叶斯网络的关键方法是图模型，构建一个图模型我们需要把具有因果联系的各个变量用箭头连在一起。贝叶斯网络的有向无环图中的节点表示随机变量。连接两个节点的箭头代表此两个随机变量是具有因果关系的。
- 贝叶斯网络是模拟人的认知思维推理模式的，用一组条件概率以及有向无环图对不确定性因果推理关系建模

最简单的一个贝叶斯网络

$$P(a, b, c) = P(c | a, b)P(b | a)P(a)$$

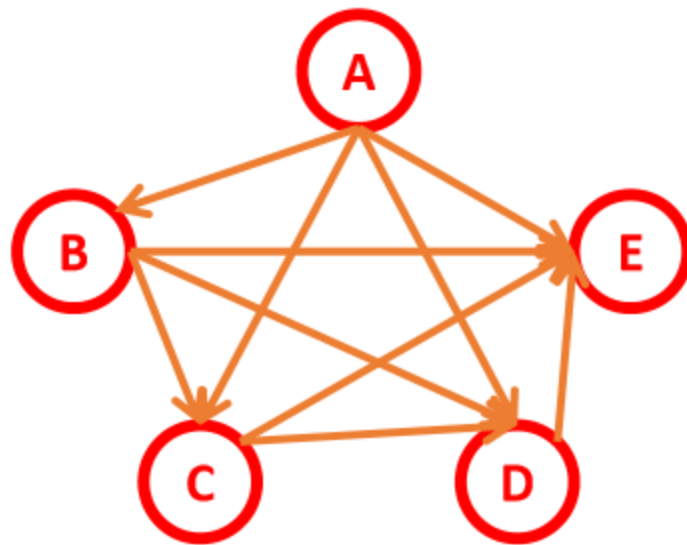


全连接贝叶斯网络

- 每一对节点之间都有边连接

$$P(x_1, x_2, \dots, x_n) = P(x_n | x_1, x_2, \dots, x_{n-1}) \dots P(x_2 | x_1) P(x_1)$$

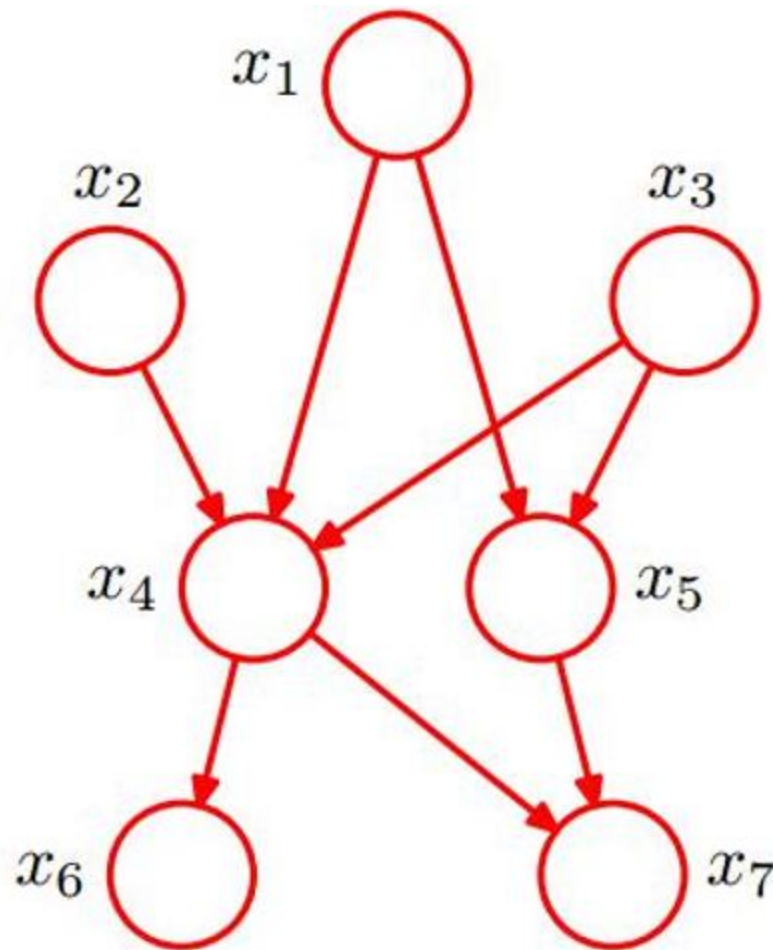
$$P(x_1, x_2, \dots, x_n) = \prod_{i=2}^n P(x_i | x_1, x_2, \dots, x_{i-1}) * P(x_1)$$



“正常” 贝叶斯网络

$$p(x_1, x_2, x_3, x_4, x_5, x_6, x_7) = p(x_1)p(x_2)p(x_3)p(x_4 | x_1, x_2, x_3)p(x_5 | x_1, x_3)p(x_6 | x_4)p(x_7 | x_4, x_5)$$

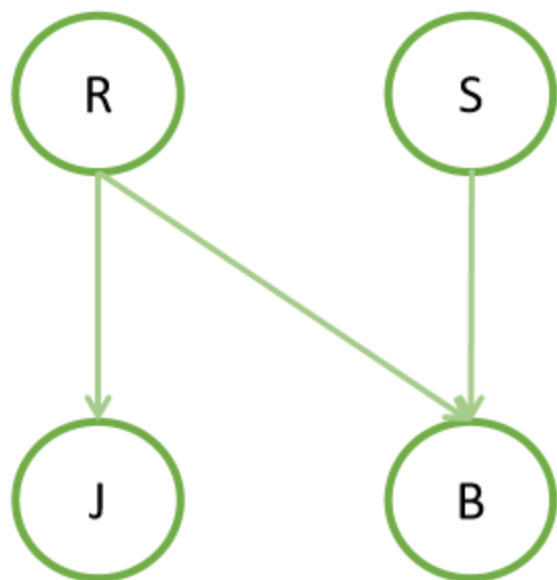
- x_1, x_2, x_3 独立
- x_6 和 x_7 在给定条件下独立
- $x_1, x_2, x_3 \dots x_7$ 的联合分布为



$R \in \{0, 1\}$ $R=1$ 表示下雨0表示没有
 $S \in \{0, 1\}$ $S=1$ 表示Bruce忘记关喷水器0表示没有忘
 $J \in \{0, 1\}$ $J=1$ 表示Joe家里的花园草地也湿了0表示没有
 $B \in \{0, 1\}$ $T=1$ 表示Bruce的花园草地湿了0表示没有

实际贝叶斯网络：判断是否下雨

- 有一天早晨，Bruce 离开他的房子的时候发现他家花园中的草地是湿的，有两种可能，第一：昨天晚上下雨了，第二：他昨天晚上忘记关掉花园中的喷水器，接下来，他观察他的邻居 Joe，发现他家花园中的草地也是湿的，因此他推断， 他家的草地湿了是因为昨天晚上下雨的缘故



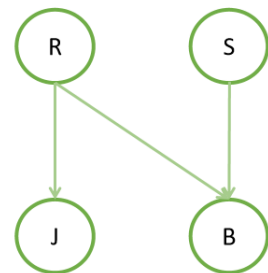
R	0	1
P(R)	0.8	0.2

R	0	1
P(J=0)	0.8	0
P(J=1)	0.2	1

S	0	1
P(S)	0.9	0.1

R	S	p(B=0)	p(B=1)
0	0	1	0
0	1	0.1	0.9
1	0	0	1
1	1	0	1

实际贝叶斯网络：判断是否下雨



R	0	1
P(R)	0.8	0.2

R	0	1
P(J=0)	0.8	0
P(J=1)	0.2	1

S	0	1
P(S)	0.9	0.1

R	S	p(B=0)	p(B=1)
0	0	1	0
0	1	0.1	0.9
1	0	0	1
1	1	0	1

$$\begin{aligned}
 p(S=1 | B=1, J=1) &= \frac{p(S=1, B=1, J=1)}{p(B=1, J=1)} \\
 &= \frac{\sum_R p(B=1, J=1, R, S=1)}{\sum_{R,S} p(B=1, J=1, R, S)} \\
 &= \frac{\sum_R p(J=1 | R) p(B=1 | R, S=1) p(R) p(S=1)}{\sum_{R,S} p(J=1 | R) p(B=1 | R, S) p(R) p(S)} \\
 &= \frac{0.0344}{0.2144} = 0.1604
 \end{aligned}$$

所以判断出应该是下雨导致草地变湿

$$\begin{aligned}
 p(R=1 | B=1, J=1) &= \frac{p(R=1, B=1, J=1)}{p(B=1, J=1)} \\
 &= \frac{\sum_S p(B=1, J=1, R=1, S)}{\sum_{R,S} p(B=1, J=1, R, S)} \\
 &= \frac{\sum_S p(J=1 | R) p(B=1 | R=1, S) p(R=1) p(S)}{\sum_{R,S} p(J=1 | R) p(B=1 | R, S) p(R) p(S)} \\
 &= \frac{0.2}{0.2144} = 0.9328
 \end{aligned}$$

贝叶斯网络判定条件独立-01

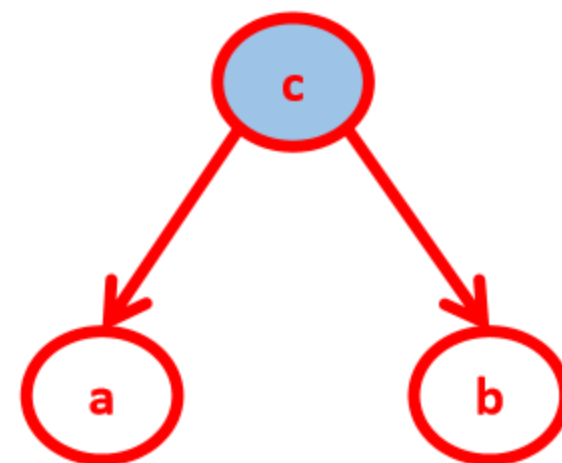
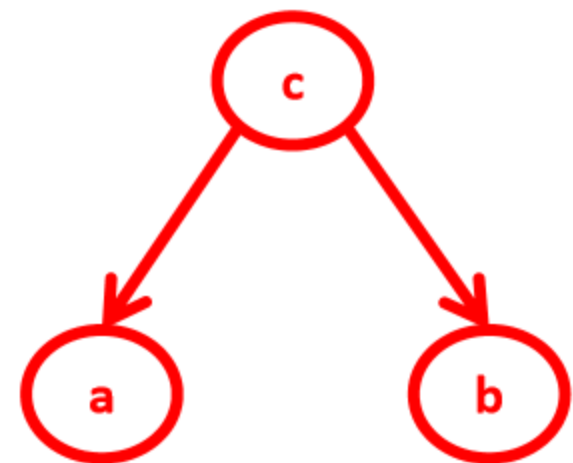
- 在C给定的条件下，a和b被阻断(blocked)是独立的
- 条件独立：tail - to -tail

$$P(a, b, c) = P(c)P(b | c)P(a | c)$$

$$\Rightarrow P(a, b, c) / P(c) = P(b | c)P(a | c)$$

$$\because p(a, b | c) = p(a, b, c) / p(c)$$

$$\therefore p(a, b | c) = p(a | c)p(b | c)$$



贝叶斯网络判定条件独立-02

- 在C给定的条件下，a和b被阻断(blocked)是独立的
- 条件独立：head- to -tail



$$p(a, b | c)$$

$$= p(a, b, c) / p(c)$$

$$P(a, b, c) = P(a)P(c | a)P(b | c)$$

$$= p(a) * p(c | a) * p(b | c) / p(c)$$

$$= p(a, c) * p(b | c) / p(c)$$

$$= p(a | c) * p(b | c)$$

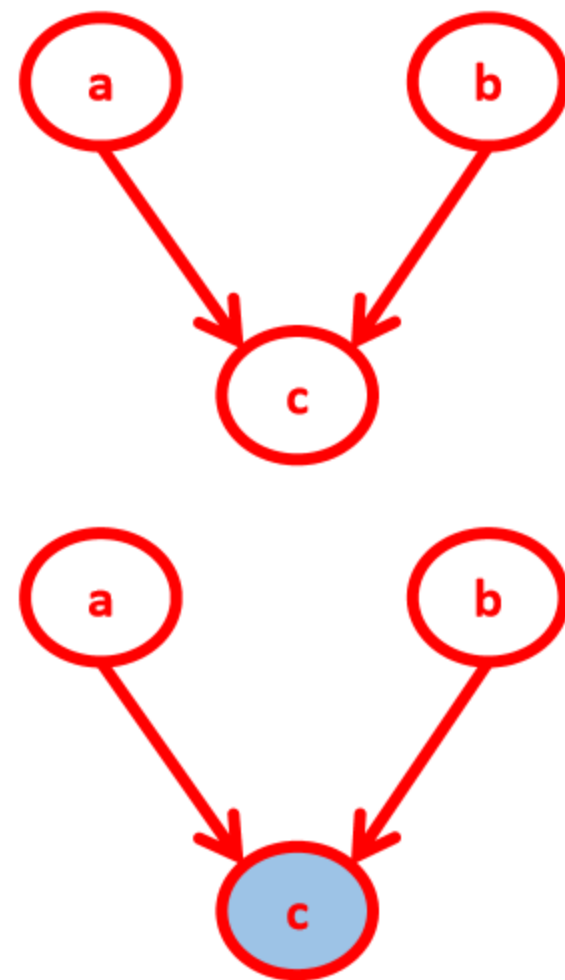
贝叶斯网络判定条件独立-03

- 在 C 未知的情况下，a 和 b 被阻断(blocked)，是独立的
- 条件独立：head - to - head

$$P(a, b, c) = P(a)P(b)P(c | a, b)$$

$$\sum_c P(a, b, c) = \sum_c P(a) * P(b) * P(c | a, b)$$

$$\Rightarrow P(a, b) = P(a) * P(b)$$

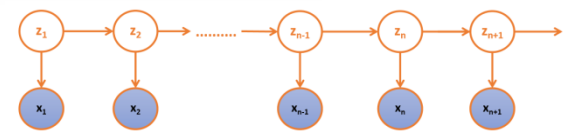


The background features a complex network diagram with numerous nodes (represented by small grey and black dots) connected by thin, light grey lines. These connections form a dense web that fills the entire frame, with some areas appearing more concentrated than others. The overall aesthetic is technical and modern.

2 chapter

隐马尔可夫模型

马尔可夫性质



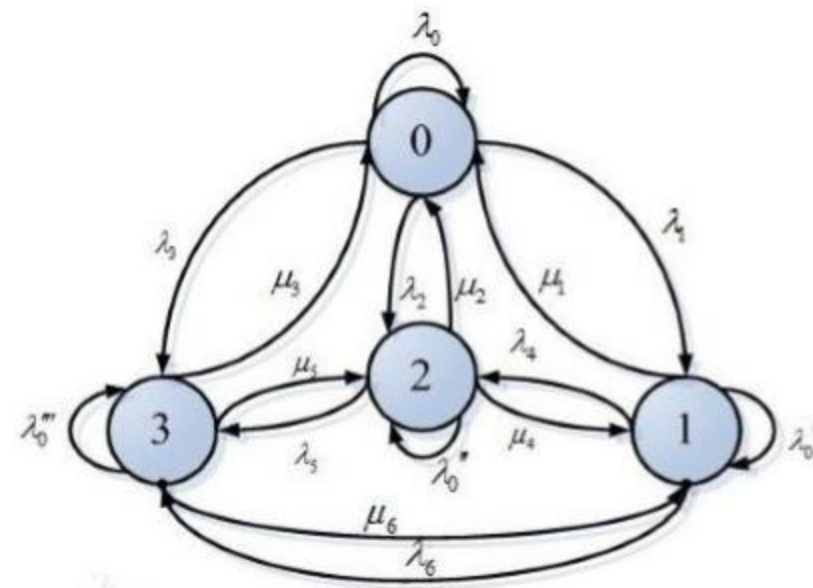
- 设 $\{X(t), t \in T\}$ 是一个随机过程， E 为其状态空间，若对于任意的 $t_1 < t_2 < \dots < t_n < t$ ，任意的 $x_1, x_2, \dots, x_n, x \in E$ ，随机变量 $X(t)$ 在已知变量 $X(t_1)=x_1, \dots, X(t_n)=x_n$ 之下的条件分布函数只与 $X(t_n)=x_n$ 有关，而与 $X(t_1)=x_1, \dots, X(t_{n-1})=x_{n-1}$ 无关，即条件分布函数满足下列等式，此性质称为马尔可夫性；如果随机过程满足马尔可夫性，则该过程称为马尔可夫过程。

$$p(X(t) \leq x | X(t_1) = x_1, \dots, X(t_n) = x_n) = p(X(t) \leq x | X(t_n) = x_n)$$

$$p(X_{n+1} = x | X_1 = x_1, \dots, X_n = x_n) = p(X_{n+1} = x | X_n = x_n)$$

马尔可夫链

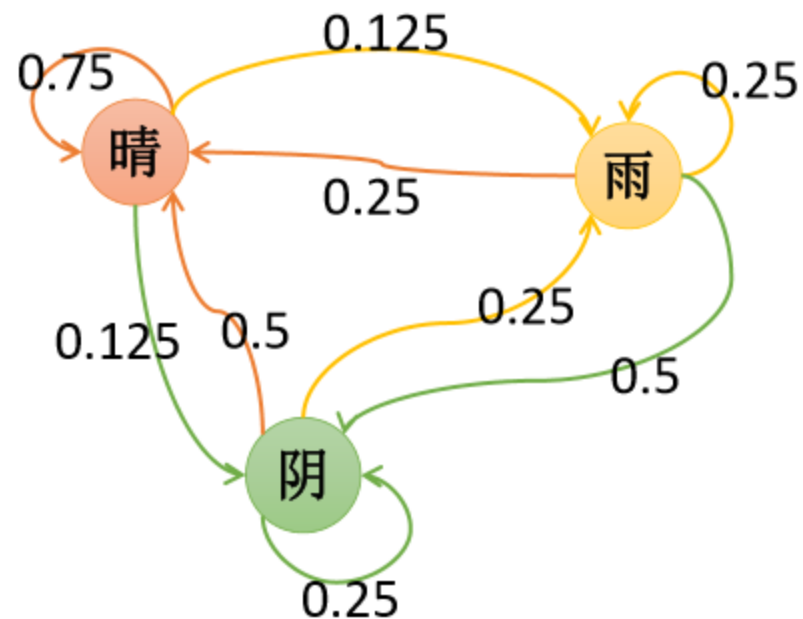
- 马尔可夫链是指具有马尔可夫性质的随机过程。在过程中，在给定当前信息的情况下，过去的信息状态对于预测将来状态是无关的。
- 在马尔可夫链的每一步，系统根据概率分布，可以从一个状态变成另外一个状态，也可以保持当前状态不变。
- 状态的改变叫做转移，状态改变的相关概率叫做转移概率。马尔可夫链中的三元素是：状态空间 S 、转移概率矩阵 P 、初始概率分布 π 。



马尔可夫链案例

- 设将天气状态分为晴、阴、雨三种状态，假定某天的天气状态只和 上一天的天气状态有关，状态使用1(晴)、2(阴)、3(雨)表示，转移 概率矩阵P如下：

今/明	晴	阴	雨
晴	0.75	0.125	0.125
阴	0.5	0.25	0.25
雨	0.25	0.5	0.25



马尔可夫链案例

- 第n+1天天气状态为j的概率为：

$$\pi(X_{n+1} = j) = \sum_{i=1}^K \pi(X_n = i) \cdot P(X_{n+1} = j | X_n = i)$$
$$\Rightarrow \pi^{n+1} = \pi^n \cdot P$$

- 因此，矩阵P即为条件概率转移矩阵。 矩阵P的第i行元素表示，在上一个状态为i的时候的分布概率，即每行元素 的和必须为1

马尔可夫链案例

■ 初始概率 $\pi[0.5,0.3,0.2]$

$$P = \begin{bmatrix} 0.75 & 0.125 & 0.125 \\ 0.5 & 0.25 & 0.25 \\ 0.25 & 0.5 & 0.25 \end{bmatrix}$$

第n天	晴	阴	雨
0	0.5	0.3	0.2
1	0.575	0.2375	0.1875
2	0.5969	0.225	0.1781
3	0.6047	0.2199	0.1754
4	0.6073	0.2183	0.1744
5	0.6082	0.2177	0.1741
6	0.6085	0.2175	0.174
7	0.6086	0.2174	0.1739
8	0.6087	0.2174	0.1739
9	0.6087	0.2174	0.1739
10	0.6087	0.2174	0.1739
11	0.6087	0.2174	0.1739

马尔可夫链案例

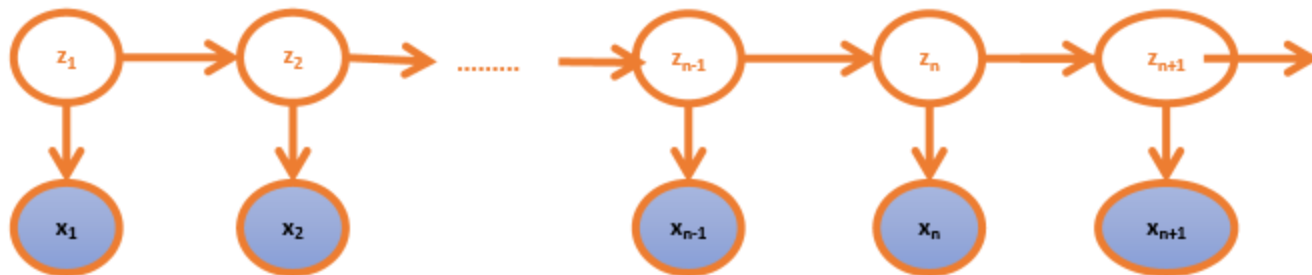
■ 初始概率 $\pi[0.1,0.6,0.3]$

$$P = \begin{bmatrix} 0.75 & 0.125 & 0.125 \\ 0.5 & 0.25 & 0.25 \\ 0.25 & 0.5 & 0.25 \end{bmatrix}$$

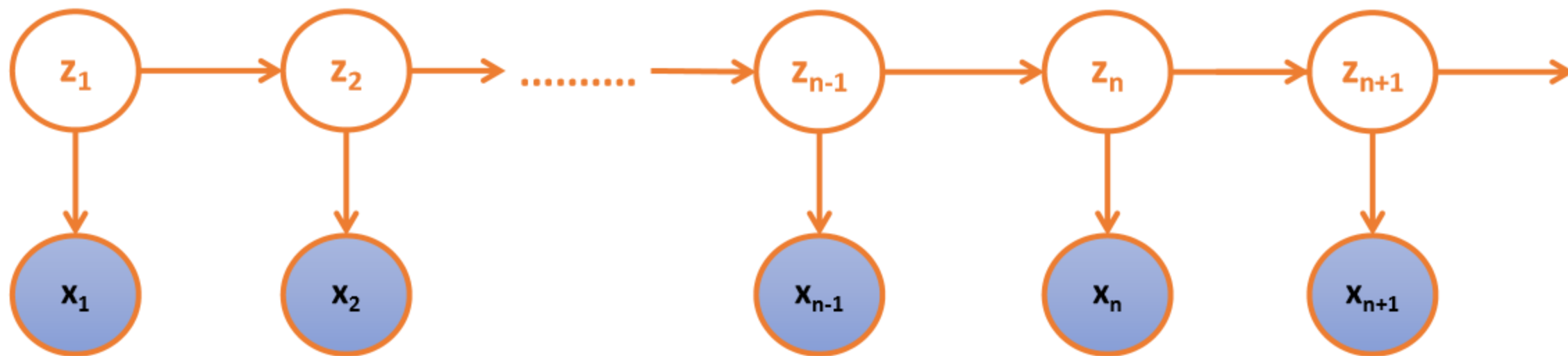
第n天	晴	阴	雨
0	0.1	0.6	0.3
1	0.45	0.3125	0.2375
2	0.5531	0.2531	0.1937
3	0.5898	0.2293	0.1809
4	0.6022	0.2215	0.1763
5	0.6065	0.2188	0.1747
6	0.6079	0.2179	0.1742
7	0.6084	0.2176	0.174
8	0.6086	0.2174	0.1739
9	0.6087	0.2174	0.1739
10	0.6087	0.2174	0.1739
11	0.6087	0.2174	0.1739

HMM

- 隐马尔可夫模型(Hidden Markov Model, HMM)是一种统计模型，在语音识别、行为识别、NLP、故障诊断等领域具有高效的性能。
- HMM是关于时序的概率模型，描述一个含有未知参数的马尔可夫链所生成的不可观测的状态随机序列，再由各个状态生成观测随机序列的过程。HMM是一个双重随机过程---具有一定状态的隐马尔可夫链和随机的观测序列。
- HMM随机生成的状态随机序列被称为状态序列；每个状态生成一个观测，由此产生的观测随机序列，被称为观测序列。



HMM



- $z_1, z_2, \dots, z_n$ 是不可观测的状态, $x_1, x_2, \dots, x_n$ 是可观测到的序列
- 在 z_1 、 z_2 不可观测的情况下, x_1 和 z_2 独立吗? x_1 和 x_2 独立吗?

HMM

- HMM由隐含状态S、可观测状态O、初始状态概率矩阵 π 、隐含状态转移概率矩阵A、发射概率矩阵B;
- π 和A决定了隐状态序列, B决定观测序列, 因此HMM可以使用三元符号表示, 称为HMM的三元素:

$$\lambda = (A, B, \pi)$$

HMM参数说明

$$\lambda = (A, B, \pi)$$

■ S是所有可能的状态集合: $S = \{s_1, s_2, \dots, s_n\}$

■ O是所有可能的观测集合: $O = \{o_1, o_2, \dots, o_m\}$

■ I是长度为T的状态序列, Q是对应的观测序列

$$I = \{i_1, i_2, \dots, i_T\} \quad Q = \{q_1, q_2, \dots, q_T\}$$

HMM参数说明

$$\lambda = (A, B, \pi)$$

- A是隐含状态转移概率矩阵:
$$A = [a_{ij}]_{n \times n} = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & \dots & \dots & \dots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix}$$
- 其中 $a_{ij} = p(i_{t+1} = s_j | i_t = s_i)$
- a_{ij} 是在时刻t处于状态 s_i 的条件下时刻t+1转移到状态 s_j 的概率。

HMM参数说明

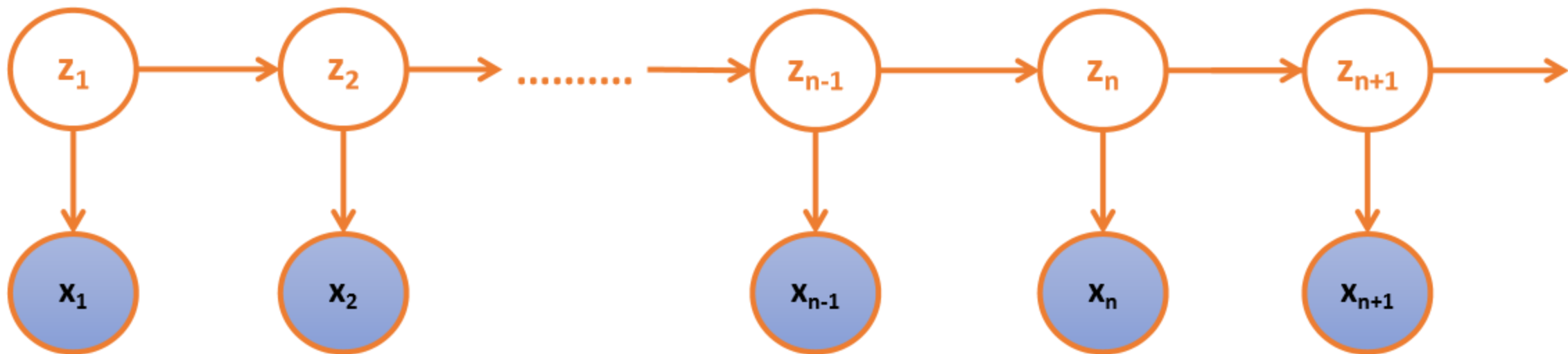
$$\lambda = (A, B, \pi)$$

- B是可观测值发射概率矩阵:

$$B = [b_{ij}]_{n \times m} = \begin{bmatrix} b_{11} & b_{12} & \dots & b_{1m} \\ b_{21} & b_{22} & \dots & b_{2m} \\ \dots & \dots & \dots & \dots \\ b_{n1} & b_{n2} & \dots & b_{nm} \end{bmatrix}$$

- 其中 $b_{ij} = p(q_t = o_j | i_t = s_i)$
- b_{ij} 是在时刻t处于状态 s_i 的条件下时刻t+1发射到状态 o_j 的概率。

HMM的两个基本性质



$$p(i_t | i_{t-1}, q_{t-1}, i_{t-2}, q_{t-2}, \dots, i_1, q_1) = p(i_t | i_{t-1})$$

$$p(q_t | i_t, i_{t-1}, q_{t-1}, i_{t-2}, q_{t-2}, \dots, i_1, q_1) = p(q_t | i_t)$$

HMM案例

- 假设有三个盒子，编号为1,2,3；每个盒子都装有黑白两种颜色的小球，球的比例如下：

编号	白球	黑球
1	4	6
2	8	2
3	5	5

- 按照下列规则的方式进行有放回的抽取小球，得到球颜色的观测序列：
 - ◆ 按照 π 的概率选择一个盒子，从盒子中随机抽取一个小球，记录颜色后，放回盒子中；
 - ◆ 按照某种条件概率选择新的盒子，重复该操作；
 - ◆ 最终得到观测序列：“白黑白白黑”；

HMM案例

- 状态集合: $S=\{\text{盒子1}, \text{盒子2}, \text{盒子3}\}$
- 观测集合: $O=\{\text{白}, \text{黑}\}$
- 状态序列和观测序列的长度 $T=5$
- 初始概率分布 π
- 状态转移概率矩阵 A
- 观测概率矩阵 B

$$\pi = \begin{pmatrix} 0.2 \\ 0.5 \\ 0.3 \end{pmatrix}$$

$$A = \begin{bmatrix} 0.5 & 0.4 & 0.1 \\ 0.2 & 0.2 & 0.6 \\ 0.2 & 0.5 & 0.3 \end{bmatrix}$$

$$B = \begin{bmatrix} 0.4 & 0.6 \\ 0.8 & 0.2 \\ 0.5 & 0.5 \end{bmatrix}$$

HMM案例思考

- 在给定参数 π 、 A 、 B 的时候，得到观测序列为“白黑白白黑”的概率是多少??

HMM的三个问题

■ 概率计算问题：前向-后向算法

◆ 给定模型 $\lambda=(A,B,\pi)$ 和观测序列 $Q=\{q_1,q_2,\dots,q_T\}$ ，计算模型 λ 下观测到序列 Q 出现的概率 $P(Q|\lambda)$

■ 学习问题：Baum-Welch算法(状态未知)

◆ 已知观测序列 $Q=\{q_1,q_2,\dots,q_T\}$ ，估计模型 $\lambda=(A,B,\pi)$ 的参数，使得在该模型下观测序列 $P(Q|\lambda)$ 最大。

■ 预测问题：Viterbi算法

◆ 给定模型 $\lambda=(A,B,\pi)$ 和观测序列 $Q=\{q_1,q_2,\dots,q_T\}$ ，求给定观测序列条件概率 $P(I|Q, \lambda)$ 最大的状态序列 I

概率计算问题

- 直接算法 暴力算法
- 前向算法
- 后向算法

直接算法

- 按照概率公式，列举所有可能的长度为T的状态序列 $I = \{i_1, i_2, \dots, i_T\}$ ，求各个状态序列 I 与观测序列 $Q = \{q_1, q_2, \dots, q_T\}$ 的联合概率 $P(Q|I; \lambda)$ ，然后对所有可能的状态序列求和，从而得到最终的概率 $P(Q; \lambda)$

$$I = \{i_1, i_2, \dots, i_T\} \quad p(I; \lambda) = \pi_{i_1} a_{i_1 i_2} a_{i_2 i_3} \dots a_{i_{T-1} i_T}$$

$$Q = \{q_1, q_2, \dots, q_T\} \quad p(Q|I; \lambda) = b_{i_1 q_1} b_{i_2 q_2} \dots b_{i_T q_T}$$

$$P(Q, I; \lambda) = p(Q|I; \lambda) p(I; \lambda) = \pi_{i_1} a_{i_1 i_2} a_{i_2 i_3} \dots a_{i_{T-1} i_T} b_{i_1 q_1} b_{i_2 q_2} \dots b_{i_T q_T}$$

$$p(Q; \lambda) = \sum_I p(Q, I; \lambda) = \sum_{i_1, i_2, \dots, i_T} \pi_{i_1} a_{i_1 i_2} a_{i_2 i_3} \dots a_{i_{T-1} i_T} b_{i_1 q_1} b_{i_2 q_2} \dots b_{i_T q_T}$$

直接算法

$$\pi = \begin{pmatrix} 0.2 \\ 0.5 \\ 0.3 \end{pmatrix}$$

$$I = \{i_1, i_2, \dots, i_T\} \quad p(I; \lambda) = \pi_{i_1} a_{i_1 i_2} a_{i_2 i_3} \dots a_{i_{T-1} i_T}$$

$$Q = \{q_1, q_2, \dots, q_T\} \quad p(Q|I; \lambda) = b_{i_1 q_1} b_{i_2 q_2} \dots b_{i_T q_T}$$

$$P(Q, I; \lambda) = p(Q|I; \lambda) p(I; \lambda) = \pi_{i_1} a_{i_1 i_2} a_{i_2 i_3} \dots a_{i_{T-1} i_T} b_{i_1 q_1} b_{i_2 q_2} \dots b_{i_T q_T}$$

$$p(Q; \lambda) = \sum_I p(Q, I; \lambda) = \sum_{i_1, i_2, \dots, i_T} \pi_{i_1} a_{i_1 i_2} a_{i_2 i_3} \dots a_{i_{T-1} i_T} b_{i_1 q_1} b_{i_2 q_2} \dots b_{i_T q_T}$$

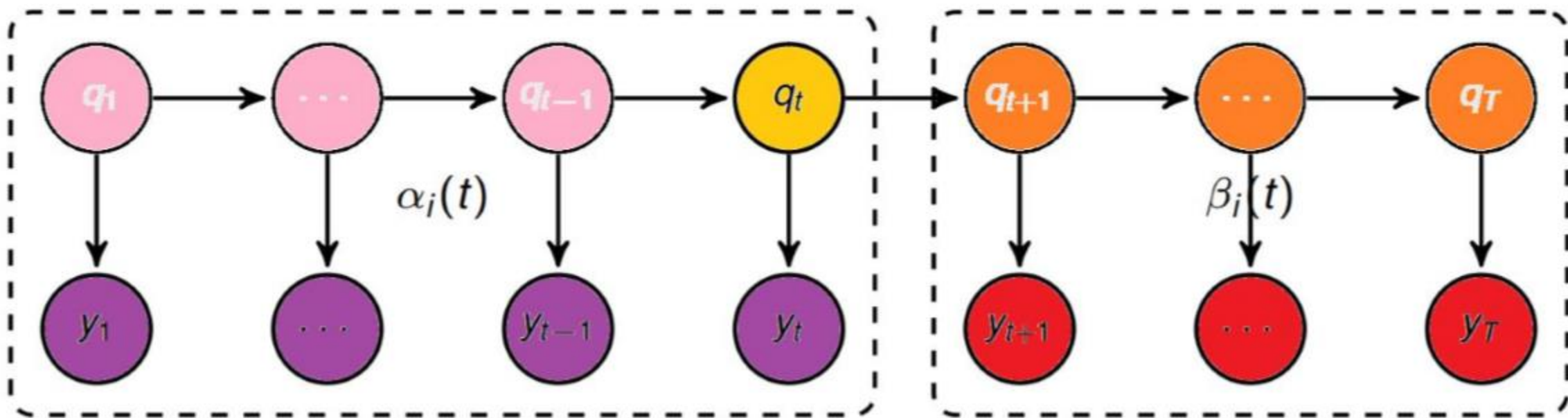
$$A = \begin{bmatrix} 0.5 & 0.4 & 0.1 \\ 0.2 & 0.2 & 0.6 \\ 0.2 & 0.5 & 0.3 \end{bmatrix}$$

$$B = \begin{bmatrix} 0.4 & 0.6 \\ 0.8 & 0.2 \\ 0.5 & 0.5 \end{bmatrix}$$

前向概率-后向概率

$$\alpha_t(i) = P(y_1, y_2, \dots, y_t, q_t = i | \lambda)$$

$$\beta_t(i) = P(y_{t+1}, y_{t+2}, \dots, y_T | q_t = i, \lambda)$$



前向算法

- 定义：给定 λ ，定义到时刻 t 部分观测序列为 $q_1, q_2, \dots, q_t$ 且状态为 s_i 的概率为前向概率。记做：

$$\alpha_t(i) = p(q_1, q_2, \dots, q_t, i_t = s_i; \lambda)$$

前向算法

- 初值: $\alpha_1(i) = p(q_1, i_1 = s_i; \lambda) = \pi_i b_{iq_1}$
- 递推: 对于 $t=1, 2, \dots, T-1$
$$\alpha_{t+1}(i) = \left(\sum_{j=1}^n \alpha_t(j) a_{ji} \right) b_{iq_{t+1}}$$
- 最终:
$$P(Q; \lambda) = \sum_{i=1}^n \alpha_T(i)$$

HMM案例-前向算法

- 在给定参数 π 、 A 、 B 的时候，得到观测序列为“白黑白白黑”的概率是多少??

$$\pi = \begin{pmatrix} 0.2 \\ 0.5 \\ 0.3 \end{pmatrix}$$

$$A = \begin{bmatrix} 0.5 & 0.4 & 0.1 \\ 0.2 & 0.2 & 0.6 \\ 0.2 & 0.5 & 0.3 \end{bmatrix}$$

$$B = \begin{bmatrix} 0.4 & 0.6 \\ 0.8 & 0.2 \\ 0.5 & 0.5 \end{bmatrix}$$

HMM案例-前向算法

$$\pi = \begin{pmatrix} 0.2 \\ 0.5 \\ 0.3 \end{pmatrix} \quad A = \begin{bmatrix} 0.5 & 0.4 & 0.1 \\ 0.2 & 0.2 & 0.6 \\ 0.2 & 0.5 & 0.3 \end{bmatrix} \quad B = \begin{bmatrix} 0.4 & 0.6 \\ 0.8 & 0.2 \\ 0.5 & 0.5 \end{bmatrix}$$

$$\alpha_1(1) = \pi_1 b_{1q_1} = 0.08$$

$$\alpha_1(2) = \pi_2 b_{2q_1} = 0.4$$

$$\alpha_1(3) = \pi_3 b_{3q_1} = 0.15$$

$$\alpha_1(i) = p(q_1, i_1 = s_i; \lambda) = \pi_i b_{iq_1}$$

$$\alpha_{t+1}(i) = \left(\sum_{j=1}^n \alpha_t(j) a_{ji} \right) b_{iq_{t+1}}$$

$$P(Q; \lambda) = \sum_{i=1}^n \alpha_T(i)$$

HMM案例-前向算法

$$\pi = \begin{pmatrix} 0.2 \\ 0.5 \\ 0.3 \end{pmatrix} \quad A = \begin{bmatrix} 0.5 & 0.4 & 0.1 \\ 0.2 & 0.2 & 0.6 \\ 0.2 & 0.5 & 0.3 \end{bmatrix} \quad B = \begin{bmatrix} 0.4 & 0.6 \\ 0.8 & 0.2 \\ 0.5 & 0.5 \end{bmatrix}$$

$$\alpha_1(1) = \pi_1 b_{1q_1} = 0.08$$

$$\alpha_1(2) = \pi_2 b_{2q_1} = 0.4$$

$$\alpha_1(3) = \pi_3 b_{3q_1} = 0.15$$

$$\alpha_2(1) = \left(\sum_{j=1}^3 \alpha_1(j) a_{j1} \right) b_{1q_2}$$

$$= (0.08 * 0.5 + 0.4 * 0.2 + 0.15 * 0.2) * 0.6$$

$$= 0.09$$

$$\alpha_2(2) = 0.0374$$

$$\alpha_2(3) = 0.1465$$

$$\alpha_3(1) = 0.032712 \quad \alpha_4(1) = 0.01702872 \quad \alpha_5(1) = 0.0142036956$$

$$\alpha_3(2) = 0.0933384 \quad \alpha_4(2) = 0.04048728 \quad \alpha_5(2) = 0.0065122938$$

$$\alpha_3(3) = 0.037695 \quad \alpha_4(3) = 0.03530505 \quad \alpha_5(3) = 0.0182933775$$

HMM案例-前向算法

$$\alpha_5(1) = 0.0142036956$$

$$\alpha_5(2) = 0.0065122938$$

$$\alpha_5(3) = 0.0182933775$$

$$p(Q; \lambda) = \sum_{i=1}^3 \alpha_5(i)$$

$$= 0.0142036956 + 0.0065122938 + 0.0182933775$$

$$= 0.0390093669$$

后向算法

- 定义：给定 λ ，定义到时刻 t 状态为 s_i 的前提下，从 $t+1$ 到 T 部分观测序列为 $q_{t+1}, q_{t+2}, \dots, q_T$ 的概率为后向概率。记做：

$$\beta_t(i) = p(q_{t+1}, q_{t+2}, \dots, q_T | i_t = s_i; \lambda)$$

后向算法

- 初值: $\beta_T(i) = 1$

- 递推: 对于 $t=T-1, T-2, \dots, 1$
$$\beta_t(i) = \sum_{j=1}^n (a_{ij} b_{jq_{t+1}} \beta_{t+1}(j))$$

- 最终:
$$P(Q; \lambda) = \sum_{i=1}^n \pi_i b_{iq_1} \beta_1(i)$$

单个状态的概率

- 求给定模型 λ 和观测序列 Q 的情况下，在时刻 t 处于状态 s_i 的概率，记做：

$$\gamma_t(i) = p(i_t = s_i | Q; \lambda)$$

- 单个状态概率的意义主要是用于判断在每个时刻最可能存在的状态，从而可以得到一个状态序列作为最终的预测结果。

单个状态的概率

$$p(i_t = s_i, Q; \lambda) = \alpha_t(i) \beta_t(i)$$

$$\gamma_t(i) = p(i_t = s_i | Q; \lambda) = \frac{p(i_t = s_i, Q; \lambda)}{p(Q; \lambda)}$$

$$\gamma_t(i) = \frac{\alpha_t(i) \beta_t(i)}{P(Q; \lambda)} = \frac{\alpha_t(i) \beta_t(i)}{\sum_{j=1}^n \alpha_t(j) \beta_t(j)}$$

两个状态的联合概率

$$\begin{aligned}\xi_t(i, j) &= p(i_t = s_i, i_{t+1} = s_j | Q; \lambda) \\ &= \frac{p(i_t = s_i, i_{t+1} = s_j, Q; \lambda)}{p(Q; \lambda)} \\ &= \frac{p(i_t = s_i, i_{t+1} = s_j, Q; \lambda)}{\sum_{i=1}^n \sum_{j=1}^n p(i_t = s_i, i_{t+1} = s_j, Q; \lambda)}\end{aligned}$$

$$p(i_t = s_i, i_{t+1} = s_j, Q; \lambda) = \alpha_t(i) a_{ij} b_{j q_{t+1}} \beta_{t+1}(j)$$

学习问题_监督学习

- 直接利用大数定理的结论“频率的极限是概率”，直接给出HMM的参数估计；

$$\hat{\pi}_i = \frac{|s_i|}{\sum_{i=1}^n |s_i|}$$

$$a_{ij} = \frac{|s_{ij}|}{\sum_{j=1}^n |s_{ij}|}$$

$$\hat{b}_{ij} = \frac{|q_{ij}|}{\sum_{j=1}^m |q_{ij}|}$$

学习问题_非监督学习

- 若训练数据中只有观测序列，则HMM的学习问题需要使用EM算法，属于非监督算法；此时一般使用Baum-Welch算法。所有的观测数据为 $Q=\{q_1, q_2, \dots, q_T\}$ ，所有的隐状态为 $I=\{i_1, i_2, \dots, i_T\}$ ，则完整的数据为 (O, I) ，完整数据的对数似然函数为 $\ln(p(Q, I; \lambda))$ ；然后直接使用EM算法的方式来进行参数估计。

Baum-Welch算法

$$p(O, I; \lambda) = \pi_{i_1} b_{i_1 q_1} a_{i_1 i_2} b_{i_2 q_2} \dots a_{i_{T-1} i_T} b_{i_T q_T}$$

$$L(\lambda, \bar{\lambda}) = \sum_I \ln(p(Q, I; \lambda)) p(I|Q; \bar{\lambda})$$

$$= \sum_I \ln(p(Q, I; \lambda)) \frac{p(I, Q; \bar{\lambda})}{p(Q; \bar{\lambda})}$$

$$\propto \sum_I \ln(p(Q, I; \lambda)) p(I, Q; \bar{\lambda})$$

$$L(\lambda, \bar{\lambda}) = \sum_I \ln(\pi_{i_1}) p(I, Q; \bar{\lambda}) + \sum_I \left(\sum_{t=1}^{T-1} \ln a_{i_t i_{t+1}} \right) p(I, Q; \bar{\lambda}) + \sum_I \left(\sum_{t=1}^T \ln b_{i_t q_t} \right) p(I, Q; \bar{\lambda})$$

Baum-Welch算法

- 极大化L函数，分别可以求得 π 、 a 、 b 的值。

$$\xi_t(i, j) = \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{\sum_{i=1}^N \sum_{j=1}^N \alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}$$

$$\gamma_t(i) = \sum_{j=1}^N \xi_t(i, j)$$

$$\pi_i = \gamma_1(i) \quad a_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)} \quad b_{ij} = \frac{\sum_{t=1, q_t=o_j}^T \gamma_t(i)}{\sum_{t=1}^T \gamma_t(i)}$$

预测问题

- 近似算法
- Viterbi算法

近似算法

- 直接在每个时刻t时候最优可能的状态作为最终的预测状态，使用下列公式计算概率值：

$$\gamma_t(i) = \frac{\alpha_t(i)\beta_t(i)}{P(Q; \lambda)} = \frac{\alpha_t(i)\beta_t(i)}{\sum_{j=1}^n \alpha_t(j)\beta_t(j)}$$

Viterbi算法

- Viterbi算法实际是用动态规划的思路求解HMM预测问题，求出概率最大的“路径”，每条“路径”对应一个状态序列。

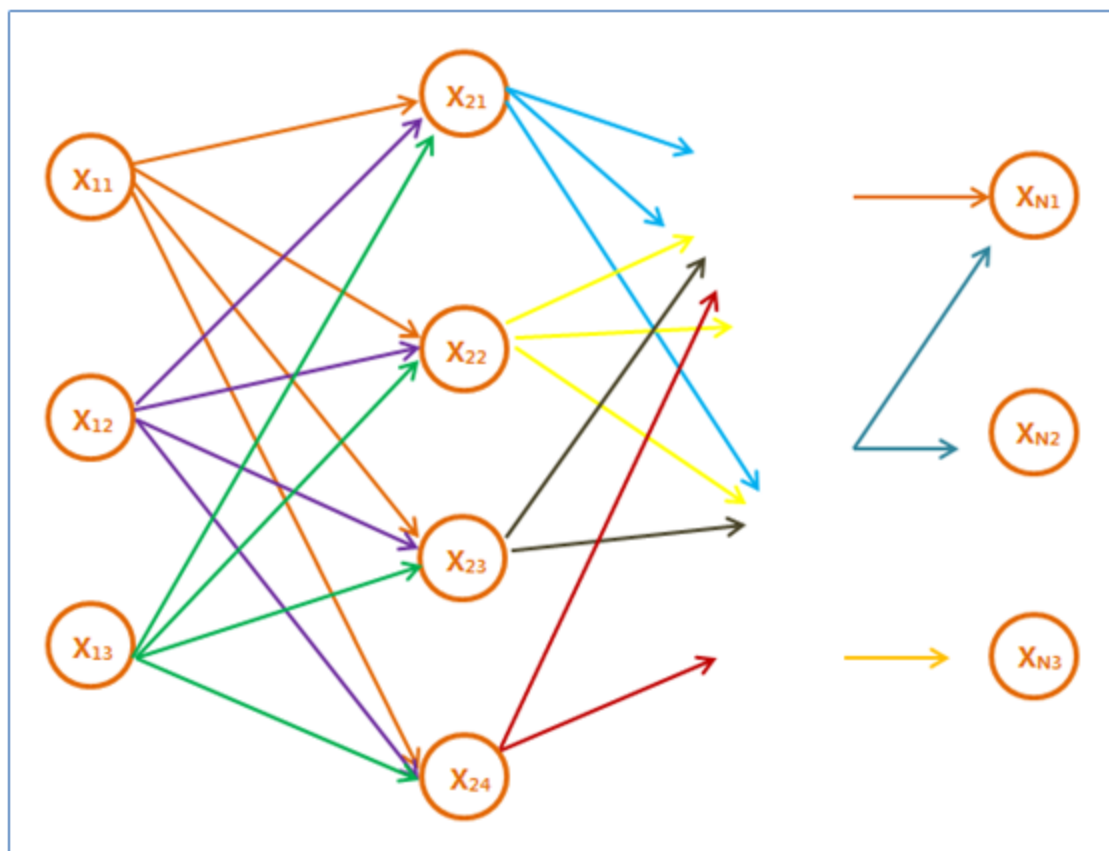
$$\delta_t(i) = \max_{i_1, i_2, \dots, i_{t-1}} p(i_t = i, i_1, i_2, \dots, i_{t-1}, q_t, q_{t-1}, \dots, q_1; \lambda)$$

$$\delta_1(i) = \pi_i b_{iq_1} \quad \delta_{t+1}(i) = \max_{1 \leq j \leq n} (\delta_t(j) a_{ji}) b_{iq_{t+1}}$$

$$P^* = \max_{1 \leq i \leq n} \delta_T(i)$$

Viterbi算法

- 维特比算法是一个特殊但应用最广的动态规划算法，它是针对篱笆网络的有向图（Lattice）的最短路径问题而提出的。凡是使用隐含马尔可夫模型描述的问题都可以用维特比算法来解码，包括今天的数字通信、语音识别、机器翻译、拼音转汉字、分词等。



维特比算法的基础

- (1) 如果概率最大的路径 P （或叫最短路径）经过某个点，比如下图中的 X_{22} ，那么这条路径上从起始点 S 到 X_{22} 的这一段子路径 Q ，一定是 S 到 X_{22} 之间的最短路径。否则，用 S 到 X_{22} 的最短路径 R 替代 Q ，便构成了一条比 P 更短的路径，这显然是矛盾的。
- (2) 从 S 到 E 的路径必定经过第 i 时刻的某个状态，假定第 i 时刻有 k 个状态，那么如果记录了从 S 到第 i 个状态的所有 k 个节点的最短路径，最终的最短路径必经过其中的一条。这样，在任何时刻，只需要考虑非常有限条最短路径即可。
- (3) 结合上述两点，假定当我们将状态 i 进入状态 $i+1$ 时，从 S 到状态 i 上各个节点的最短路径已经找到，并且记录在这些节点上，那么在计算从起点 S 到前一个状态 i 所有的 k 个节点的最短路径，以及从这 k 个节点到 $X_{i+1, j}$ 的距离即可。

维特比算法案例

■ 1、问题描述

假设连续观察3天的海藻湿度为(Dry,Damp,Soggy),求这三天最可能的天气情况。

■ 2、已知信息

- ◆ ①天气只有三类(Sunny,Cloudy,Rainy), 海藻湿度有四类{Dry,Dryish, Damp,Soggy }, 而且海藻湿度和天气有一定的关系。
- ◆ ②隐藏的状态: Sunny, Cloudy, Rainy;
- ◆ ③观察状态序列: {Dry, Damp, Soggy}

维特比算法案例

④初始状态序列：

Sunny	Cloudy	Rainy
0.63	0.17	0.20

⑤状态转移矩阵：

	Sunny	Cloudy	Rainy
Sunny	0.5	0.375	0.125
Cloudy	0.25	0.125	0.625
Rainy	0.25	0.375	0.375

⑥发射矩阵：

	Dry	Dryish	Damp	Soggy
Sunny	0.6	0.2	0.15	0.05
Cloudy	0.25	0.25	0.25	0.25
Rainy	0.05	0.10	0.35	0.5

维特比算法案例

■ 3、分析

由一阶HMM可知, Day2的天气仅取决于Day1; Day3的天气又只取决于Day2的天气。

■ 4、计算过程

(1) Day1由于是初始状态, 我们分别求

$$P(\text{Day1-Sunny})=0.63*0.6;$$

$$P(\text{Day1-Cloudy})=0.17*0.25;$$

$$P(\text{Day1-Rain})=0.20*0.05;$$

Choose $\max\{P(\text{Day1-Sunny}), P(\text{Day1-Cloudy}), P(\text{Day1-Rainy})\}$, 得到 $P(\text{Day1-Sunny})$ 最大, 得出第1天Sunny的概率最大。

维特比算法案例

(2) Day2的天气又取决于Day1的天气状况,同时也受Day2观察的海藻情况影响。

$$P(\text{Day2-Sunny}) = \max\{P(\text{Day1-Sunny}) * 0.5, P(\text{Day1-Cloudy}) * 0.25, P(\text{Day1-Rainy}) * 0.25\} * 0.15;$$

$$P(\text{Day2-Cloudy}) = \max\{P(\text{Day1-Sunny}) * 0.375, P(\text{Day1-Cloudy}) * 0.125, P(\text{Day1-Rainy}) * 0.625\} * 0.25;$$

$$P(\text{Day2-Rainy}) = \max\{P(\text{Day1-Sunny}) * 0.125, P(\text{Day1-Cloudy}) * 0.625, P(\text{Day1-Rainy}) * 0.375\} * 0.35;$$

Choose $\max\{P(\text{Day2-Sunny}), P(\text{Day2-Cloudy}), P(\text{Day2-Rainy})\}$, 得到 $P(\text{Day2-Rainy})$ 最大, 得出第2天 Rainy 的概率最大。

故 {Sunny, Rainy} 是前两天最大可能的天气序列。

维特比算法案例

(3) Day3的天气又取决于Day2的天气状况，同时也受Day3观察的海藻情况影响。

$$P(\text{Day3-Sunny}) = \max\{P(\text{Day2-Sunny}) * 0.5, P(\text{Day2-Cloudy}) * 0.25, P(\text{Day2-Rainy}) * 0.25\} * 0.05;$$

$$P(\text{Day3-Cloudy}) = \max\{P(\text{Day2-Sunny}) * 0.375, P(\text{Day2-Cloudy}) * 0.125, P(\text{Day2-Rainy}) * 0.625\} * 0.25;$$

$$P(\text{Day3-Rainy}) = \max\{P(\text{Day2-Sunny}) * 0.125, P(\text{Day2-Cloudy}) * 0.625, P(\text{Day2-Rainy}) * 0.375\} * 0.05;$$

Choose $\max\{P(\text{Day3-Sunny}), P(\text{Day3-Cloudy}), P(\text{Day3-Rainy})\}$, 得到 $P(\text{Day3-Rainy})$ 最大，得出第3天Rainy的概率最大。故 {Sunny, Rainy, Rainy} 是这三天最可能的天气序列。

HMM案例-Viterbi

- 在给定参数 π 、 A 、 B 的时候，得到观测序列为“白黑白白黑”，求出最优的隐藏状态序列。

$$\pi = \begin{pmatrix} 0.2 \\ 0.5 \\ 0.3 \end{pmatrix}$$

$$A = \begin{bmatrix} 0.5 & 0.4 & 0.1 \\ 0.2 & 0.2 & 0.6 \\ 0.2 & 0.5 & 0.3 \end{bmatrix}$$

$$B = \begin{bmatrix} 0.4 & 0.6 \\ 0.8 & 0.2 \\ 0.5 & 0.5 \end{bmatrix}$$

HMM案例-Viterbi

$$\delta_1(i) = \pi_i b_{iq_1} = \pi_i b_{i\text{白}}$$
$$\begin{aligned}\delta_1(1) &= 0.08 \\ \delta_1(2) &= 0.4 \\ \delta_1(3) &= 0.15\end{aligned}$$

$$\delta_2(i) = \max_{1 \leq j \leq 3} (\delta_1(j) a_{ji}) b_{iq_2} = \max_{1 \leq j \leq 3} (\delta_1(j) a_{ji}) b_{i\text{黑}}$$

$$\delta_2(1) = \max\{0.08 * 0.5, 0.4 * 0.2, 0.15 * 0.2\} * 0.6 = 0.048$$

$$\delta_2(2) = \max\{0.08 * 0.4, 0.4 * 0.2, 0.15 * 0.5\} * 0.2 = 0.016$$

$$\delta_2(3) = \max\{0.08 * 0.1, 0.4 * 0.6, 0.15 * 0.3\} * 0.5 = 0.12$$

HMM案例-Viterbi

$$\begin{array}{ll}\delta_1(1) = 0.08 & \delta_2(1) = 0.048 \\ \delta_1(2) = 0.4 & \delta_2(2) = 0.016 \\ \delta_1(3) = 0.15 & \delta_2(3) = 0.12\end{array}$$

$$\begin{array}{lll}\delta_3(1) = 0.0096 & \delta_4(1) = 0.00384 & \delta_5(1) = 0.001728 \\ \delta_3(2) = 0.048 & \delta_4(2) = 0.00768 & \delta_5(2) = 0.00144 \\ \delta_3(3) = 0.018 & \delta_4(3) = 0.0144 & \delta_5(3) = 0.002304\end{array}$$

■ 最终盒子序列为: (2, 3, 2, 2, 3)

下面我们通过numpy实现HMM模型并完成词性标注任务，该案例共有HMM类和主函数两个部分，其中HMM类包含以下几个方法：

- `__init__`: 初始化，接收一个参数`min_count`（最小词频参数，解决未登录词问题）
- `train`: 完成hmm训练，接收多个词序列（x）和标注序列（y）
- `predict`: 完成预测，接收多个词序列（x）
- `decoding`: 完成一个句子得预测，接收一个词序列

```
import numpy as np
from collections import defaultdict
class HMM(object):
    def __init__(self,min_count=5):
        """Args:
            N: 状态数，这里对应存在的标注的种类
            M: 观测数，这里对应有多少不同的字
        """
        self.min_count=min_count
```

实现train方法:

1. 统计词频

```
11 def train(self, word_lists, tag_lists):
12     """HMM的训练, 即根据训练语料对模型参数进行估计,
13     因为我们有观测序列以及其对应的状态序列, 所以我们
14     可以使用极大似然估计的方法来估计隐马尔可夫模型的参数
15     参数:
16     word_lists: 列表, 其中每个元素由字组成的列表, 如 ['担', '任', '科', '员']
17     tag_lists: 列表, 其中每个元素是由对应的标注组成的列表, 如 ['O', 'O', 'B-TITLE', 'E-TITLE']
18     """
19     #输入词序列长度和输出标签序列长度必须相等
20     assert len(tag_lists) == len(word_lists)
21
22     vocab=defaultdict(int)
23     tags=defaultdict(int)
24     for s,p in zip(word_lists, tag_lists): #统计词频, 标签频率
25         for w,t in zip(s,p):
26             vocab[w]+=1
27             tags[t]+=1
```

实现train方法:

2. 创建字典

```
28     #过滤最小词频
29     vocab={ k:v for k,v in vocab.items() if v>=self.min_count}
30     vocab=sorted(vocab.items(),key=lambda x:x[1], reverse=False)
31     tags=sorted(tags.items(),key=lambda x:x[1], reverse=False)
32     self.word2id={k:i+1 for i, (k,v) in enumerate(vocab)} #转成字典
33     self.word2id['<unk>']=0 #未登录词
34     self.tag2id={k:i for i, (k,v) in enumerate(tags)}
35     self.id2tag={v:k for k,v in self.tag2id.items()}
```

实现train方法:

3. 训练数据转成id并初始化参数

HMM的三个要素可以解释为，初始状态分布就是每一个标注作为句子第一个字的标注的概率，状态转移概率矩阵就是由某一个标注转移到下一个标注的概率（设状态转移矩阵为 M ，那么若前一个词的标注为 tag_i ，则下一个词的标注为 tag_j 的概率为 M_{ij} ），观测概率矩阵就是指在某个标注下，生成某个词的概率。根据HMM的三个要素，我们可以定义如下的HMM模型:

```
36     #处理数据将词映射成id
37     word_lists=[[self.word2id[w] if w in self.word2id.keys() else self.word2id['<unk>'] for w in s] for s in word_lists]
38     tag_lists=[[self.tag2id[t] for t in p] for p in tag_lists]
39     self.N=len(self.tag2id) #N:状态数, 这里对应存在的标注的种类
40     self.M =len(self.word2id) #M:观测数, 这里对应有多少不同的字
41     #状态转移概率矩阵A[i][j]表示从i状态转移到状态j的概率
42     self.A=np.zeros([self.N, self.N])
43     #观测概率矩阵, B[i][j]表示i状态下生成j观测的概率
44     self.B=np.zeros([self.N, self.M])
45     #初始状态概率Pi表示初始时刻为状态i的概率
46     self.Pi=np.zeros(self.N)
```


实现train方法:

4. 学习参数。HMM模型的训练过程对应隐马尔可夫模型的学习问题，实际上就是根据训练数据根据最大似然的方法估计模型的三个要素，即上文提到的初始状态分布、状态转移概率矩阵以及观测概率矩阵。

```

48     # 估计转移概率矩阵
49     for tag_list in tag_lists:
50         seq_len = len(tag_list) # 序列长度
51         for i in range(seq_len - 1):
52             current_tagid = tag_list[i] # 获取当前标签id
53             next_tagid = tag_list[i+1] # 获取下个时刻标签id
54             self.A[current_tagid][next_tagid] += 1 # 频率+1
55     # 问题: 如果某元素没有出现过, 该位置为0, 这在后续的计算中是不允许的
56     # 解决方法: 我们将等于0的概率加上很小的数
57     self.A[self.A == 0.] = 1e-10 # 概率不能为0
58     self.A = self.A / self.A.sum(axis=1, keepdims=True) #  $p(a, b)/p(a)$ 
59
60     # 估计观测概率矩阵
61     for tag_list, word_list in zip(tag_lists, word_lists):
62         assert len(tag_list) == len(word_list)
63         for tag, word in zip(tag_list, word_list):
64             self.B[tag][word] += 1 # 频率+1
65     self.B[self.B == 0.] = 1e-10 # 概率不能为0
66     self.B = self.B / self.B.sum(axis=1, keepdims=True) #  $p(a, b)/p(a)$ 

```

```

68     # 估计初始状态概率
69     for tag_list in tag_lists:
70         init_tagid = tag_list[0]
71         self.Pi[init_tagid] += 1
72     self.Pi[self.Pi == 0.] = 1e-10
73     self.Pi = self.Pi / self.Pi.sum()

```

实现decoding方法:

模型训练完毕之后, 要利用训练好的模型进行解码, 就是对给定的模型未见过的句子, 求句子中的每个字对应的标注, 针对这个解码问题, 我们使用的是维特比 (viterbi) 算法

```
84     def decoding(self, word_list):
85         """
86         使用维特比算法对给定观测序列求状态序列, 这里就是对字组成的序列, 求其对应的标注。
87         维特比算法实际是用动态规划解隐马尔可夫模型预测问题, 即用动态规划求概率最大路径 (最优路径)
88         这时一条路径对应着一个状态序列
89         """
90         # 问题: 整条链很长的情况下, 十分多的小概率相乘, 最后可能造成下溢
91         # 解决办法: 采用对数概率, 这样源空间中的很小概率, 就被映射到对数空间的大的负数
92         # 同时相乘操作也变成简单的相加操作
93         A = np.log(self.A)
94         B = np.log(self.B)
95         Pi = np.log(self.Pi)
96
97         # 初始化 维比特矩阵viterbi 它的维度为[状态数, 序列长度]
98         # 其中viterbi[i, j]表示标注序列的第j个标注为i的所有单个序列(i_1, i_2, .. i_j)出现的概率最大值
99         seq_len = len(word_list)
100         viterbi = np.zeros([self.N, seq_len])
101         # backpointer是跟viterbi一样大小的矩阵
102         # backpointer[i, j]存储的是 标注序列的第j个标注为i时, 第j-1个标注的id
103         # 等解码的时候, 我们用backpointer进行回溯, 以求出最优路径
104         backpointer = np.zeros([self.N, seq_len], dtype=np.int32)
```

实现decoding方法:

```
106     # 所以第一步为
107     start_wordid = word_list[0]
108     Bt = B.T
109     bt = Bt[start_wordid]
110     viterbi[:, 0] = Pi + bt
111     backpointer[:, 0] = -1
112
113     # 递推公式:
114     # viterbi[tag_id, step] = max(viterbi[:, step-1]* self.A.t()[tag_id] * Bt[word])
115     # 其中word是step时刻对应的字
116     # 由上述递推公式求后续各步
117     for step in range(1, seq_len):
118         wordid = word_list[step]
119         # 处理字不在字典中的情况
120         # bt是在t时刻字为wordid时, 状态的概率分布
121         if wordid is None:
122             # 如果字不再字典里, 则假设状态的概率分布是均匀的
123             bt = np.log(np.ones(self.N) / self.N)
124         else:
125             bt = Bt[wordid] # 否则从观测概率矩阵中取bt
126         for tag_id in range(len(self.tag2id)):
127             max_prob = np.max(
128                 viterbi[:, step-1] + A[:, tag_id],
129                 axis=0
130             )
131             max_id = np.argmax(
132                 viterbi[:, step-1] + A[:, tag_id],
133                 axis=0
134             )
135             viterbi[tag_id, step] = max_prob + bt[tag_id]
136             backpointer[tag_id, step] = max_id
```

实现decoding方法:

```
138     # 终止, t=seq_len 即 viterbi[:, seq_len]中的最大概率, 就是最优路径的概率
139     best_path_prob= np.max(
140         viterbi[:, seq_len-1], axis=0
141     )
142     best_path_pointer= np.argmax(
143         viterbi[:, seq_len-1], axis=0
144     )
145
146     # 回溯, 求最优路径
147     best_path_pointer = best_path_pointer.item()
148     best_path = [best_path_pointer]
149     for back_step in range(seq_len-1, 0, -1):
150         best_path_pointer = backpointer[best_path_pointer, back_step]
151         best_path_pointer = best_path_pointer.item()
152         best_path.append(best_path_pointer)
153
154     # 将tag_id组成的序列转化为tag
155     assert len(best_path) == len(word_list)
156
157     tag_list = [self.id2tag[id_] for id_ in reversed(best_path)]
158
159     return tag_list
```

最后，实现predict方法：

```
75     def predict(self, word_lists):
76         word_lists=[[self.word2id[w] if w in self.word2id.keys() else self.word2id['<unk>'] for w in s] for s in word_lists]
77
78         pred_tag_lists = []
79         for word_list in word_lists:
80             pred_tag_list = self.decoding(word_list)
81             pred_tag_lists.append(pred_tag_list)
82         return pred_tag_lists
```

编写主函数进行训练和预测：

1. 加载训练集数据

```

1 import json
2 filename='data/pos_train.json' #训练数据文件，存储一个列表的句子，一个句子是一个（词，词性）对
3 print("开始加载数据.....")
4 with open(filename,encoding='utf-8') as f:
5     train_sents=json.load(f)
6 train_sentences=[]
7 train_pos=[]
8 for sent in train_sents:
9     s=[]
10    p=[]
11    for w in sent:
12        s.append(w[0])
13        p.append(w[1])
14    train_sentences.append(s)
15    train_pos.append(p)
16 print(train_sentences[:2],train_pos[:2])

```

开始加载数据.....

[['必须', '指出', ' ', ' ', ' ', ' ', '长期', '共存', ' ', ' ', '互相', '监督', ' ', ' ', ' ', '的', ' ', '方针', ' ', '是',
 '各', ' ', '民主党派', ' ', '是否', ' ', '能够', ' ', '长期', ' ', '存在', ' ', '下去', ' ', ' ', ' ', '这', ' ', '不是', ' ', '由', ' ', '共产',
 ' ', ' ', ' ', '还要', ' ', '看', ' ', '各', ' ', '民主党派', ' ', '自己', ' ', '的', ' ', '表现', ' ', ' ', ' ', '看', ' ', '它们', ' ', '是不是',
 '导', ' ', ' ', ' ', '走', ' ', '社会主义', ' ', '道路', ' ', '来', ' ', '决定', ' ', ' ', ' ', '这', ' ', '是', ' ', '实现', ' ', '长期', ' ', '共',
 '作品', ' ', '今', ' ', '亦', ' ', '不', ' ', '存', ' ', ' ', ' ', '但', ' ', '从', ' ', '《', ' ', '院本', ' ', '名目', ' ', '》', ' ', '看', ' ', ' ',
 '名目', ' ', '几乎', ' ', '减', ' ', '得', ' ', '不', ' ', '剩', ' ', '几个', ' ', ' ', ' ', '而', ' ', ' ', ' ', '引', ' ', '首', ' ', ' ', ' ', ' ',
 ' ', ' ', '等', ' ', '形式', ' ', '则', ' ', '明显', ' ', '增多', ' ', ' ', ' ', '日趋', ' ', '重要', ' ', ' ', ' ', '更', ' ', '值得', ' ', '注意',
 '把', ' ', '说唱', ' ', '故事', ' ', '与', ' ', '滑稽', ' ', '表演', ' ', '糅合', ' ', '到', ' ', '一起', ' ', ' ', ' ', '使', ' ', '戏曲', ' ', '特',
 'v', 'w', 'w', 'nt', 'v', 'w', 'd', 'v', 'w', 'u', 'n', 'vl', 'n', 'a', 'u', 'n', 'w', 'v',
 'r', 'vl', 'p', 'n', 'n', 'u', 'n', 'u', 'v', 'w', 'vu', 'v', 'r', 'n', 'r', 'u', 'n', 'v',
 'n', 'w', 'v', 'n', 'n', 'vd', 'v', 'w', 'r', 'vl', 'v', 'nt', 'v', 'u', 'a', 'n', 'w', 'l',
 'w', 'n', 'n', 'w', 'v', 'w', 'n', 'v', 'r', 'n', 'n', 'u', 'n', 'd', 'v', 'u', 'd', 'v',
 'a', 'n', 'w', 'w', 'w', 'n', 'n', 'w', 'u', 'n', 'c', 'a', 'v', 'w', 'd', 'a', 'w', 'd',
 'p', 'n', 'n', 'c', 'a', 'v', 'v', 'v', 'n', 'w', 'v', 'n', 'n', 'v', 'u', 'd', 'a', 'w',

编写主函数进行训练和预测：

2. 实例化hmm模型并训练

```
1 hmm=HMM()  
2 hmm.train(train_sentences, train_pos)
```

3. 加载测试集数据

```
1 filename='data/pos_test.json' #训练数据文件，存储一个列表的句子，一个句子是一个（词，词性）对  
2 print("开始加载测试数据.....")  
3 with open(filename, encoding='utf-8') as f:  
4     train_sents=json.load(f)  
5 sentences=[]  
6 pos=[]  
7 for sent in train_sents:  
8     s=[]  
9     p=[]  
10    for w in sent:  
11        s.append(w[0])  
12        p.append(w[1])  
13    sentences.append(s)  
14    pos.append(p)
```

编写主函数进行训练和预测：

4. 使用训练得hmm模型预测

```
1 #开始测试
2 y_pred=hmm.predict(sentences[:2])
3 result=[[ w+'/' +p+'/' +t for w,p,t in zip(sent,y_p,pos)] for sent,y_p,pos in zip(sentences[:2],y_pred,pos[:2])]
4 print("结果(词/预测词性/真实词性/)：" ,result)
```

结果如下，可以看出，错误得很少：

结果(词/预测词性/真实词性/)： [['必须/d/d', '指出/v/v', ', /w/w', '"/w/w', '长期/nt/nt', '共存/v/v', '、/w/w', '互相/d/d', '监督/v/v', '"/w/w', '的/u/u', '方针/n/n', '是/vl/vl', '我党/n/n', '既定/a/a', '的/u/u', '政策/n/n', ', /w/w', '至于/p/v', '各/r/r', '民主党派/n/n', '是否/d/d', '能够/vu/vu', '长期/nt/nt', '存在/v/v', '下去/vd/vd', ', /w/w', '这/r/r', '不是/vl/vl', '由/p/p', '共产党/n/n', '一方面/n/n', '的/u/u', '愿望/n/n', '所/u/u', '决定/v/v', ', /w/w', '还要/vu/vu', '看/v/v', '各/r/r', '民主党派/n/n', '自己/r/r', '的/u/u', '表现/n/n', ', /w/w', '看/v/v', '它们/r/r', '是不是/v/v', '诚心诚意/i/i', '地/u/u', '接受/v/v', '共产党/n/n', '的/u/u', '领导/n/n', '、/w/w', '走/v/v', '社会主义/n/n', '道路/n/n', '来/vd/vd', '决定/v/v', ', /w/w', '这/r/r', '是/vl/vl', '实现/v/v', '长期/nt/nt', '共存/v/v', '的/u/u', '根本/a/a', '条件/n/n', '。/w/w'], ['金院本/nh/n', '作品/n/n', '今/nt/nt', '亦/d/d', '不/d/d', '存/v/v', ', /w/w', '但/c/c', '从/p/p', '《/w/w', '院本/v/n', '名目/n/n', '》/w/w', '看/v/v', ', /w/w', '全部/n/n', '袭用/v/v', '其它/r/r', '艺术/n/n', '样式/n/n', '的/u/u', '名目/n/n', '几乎/d/d', '减/v/v', '得/u/u', '不/d/d', '剩/v/v', '几个/mq/mq', ', /w/w', '而/c/c', '"/w/w', '引/v/v', '首/n/n', '"/w/w', '、/w/w', '"/w/w', '艳/a/a', '段/q/n', '"/w/w', '、/w/w', '"/w/w', '栓/nh/n', '摘/v/n', '"/w/w', '等/u/u', '形式/n/n', '则/c/c', '明显/a/a', '增多/v/v', ', /w/w', '日趋/d/d', '重要/a/a', '；/w/w', '更/d/d', '值得/v/v', '注意/v/v', '的/u/u', '是/vl/vl', '"/w/w', '院/n/n', '么/n/a', '"/w/w', '形式/n/n', '把/p/p', '说唱/n/n', '故事/n/n', '与/c/c', '滑稽/a/a', '表演/v/v', '糅合/n/v', '到/v/v', '一起/n/n', ', /w/w', '使/v/v', '戏曲/n/n', '特点/n/n', '表现/v/v', '得/u/u', '更/d/d', '突出/a/a', '。/w/w']]



Q&A

TRANSWARP
星环科技